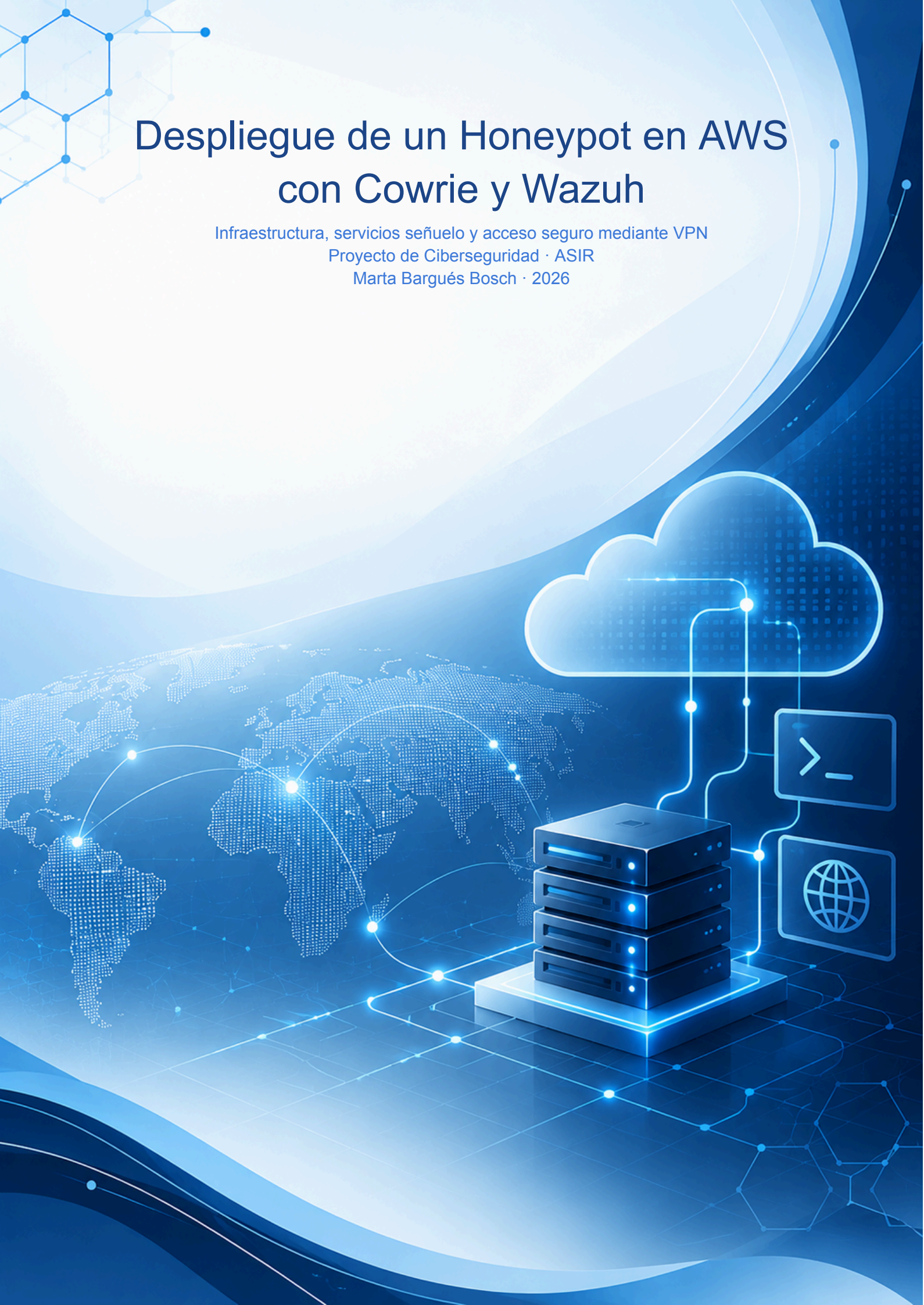
Despliegue de un Honeypot en AWS con Cowrie y Wazuh

Infraestructura, servicios señuelo y acceso seguro mediante VPN

Proyecto de Ciberseguridad · ASIR

Marta Bargués Bosch · 2026



1. Introducción **3**
1.1 Objetivo 3
2. Arquitectura del laboratorio **3**
3. Despliegue de la infraestructura en AWS **4**
3.1 Diseño de la red 4
3.2 Enrutamiento y acceso a Internet 5
3.3 Control del tráfico 5
3.4 Acceso mediante VPN 6
4. Despliegue y configuración del honeypot **6**
4.1 Instalación y configuración de Nginx 6
4.2 Configuración DNS 7
4.3 HTTPS y certificado TLS 8
4.4 Web señuelo Aegis Bank 8
5. Despliegue del honeypot SSH con Cowrie **8**
5.1 Instalación y configuración de Cowrie 8
5.2 Separación del SSH real y el servicio señuelo 9
5.3 Redirección del puerto 22 hacia Cowrie 9
5.4 Persistencia y arranque automático 9
5.5 Validación desde Internet 10
6. Integración con Wazuh **10**
6.1 Servidor Wazuh local 10
6.2 Comunicación entre AWS y Wazuh 11
6.3 Conexión del agente Wazuh 12
7. Resultado final **13**
8. Conclusiones **13**

1. Introducción

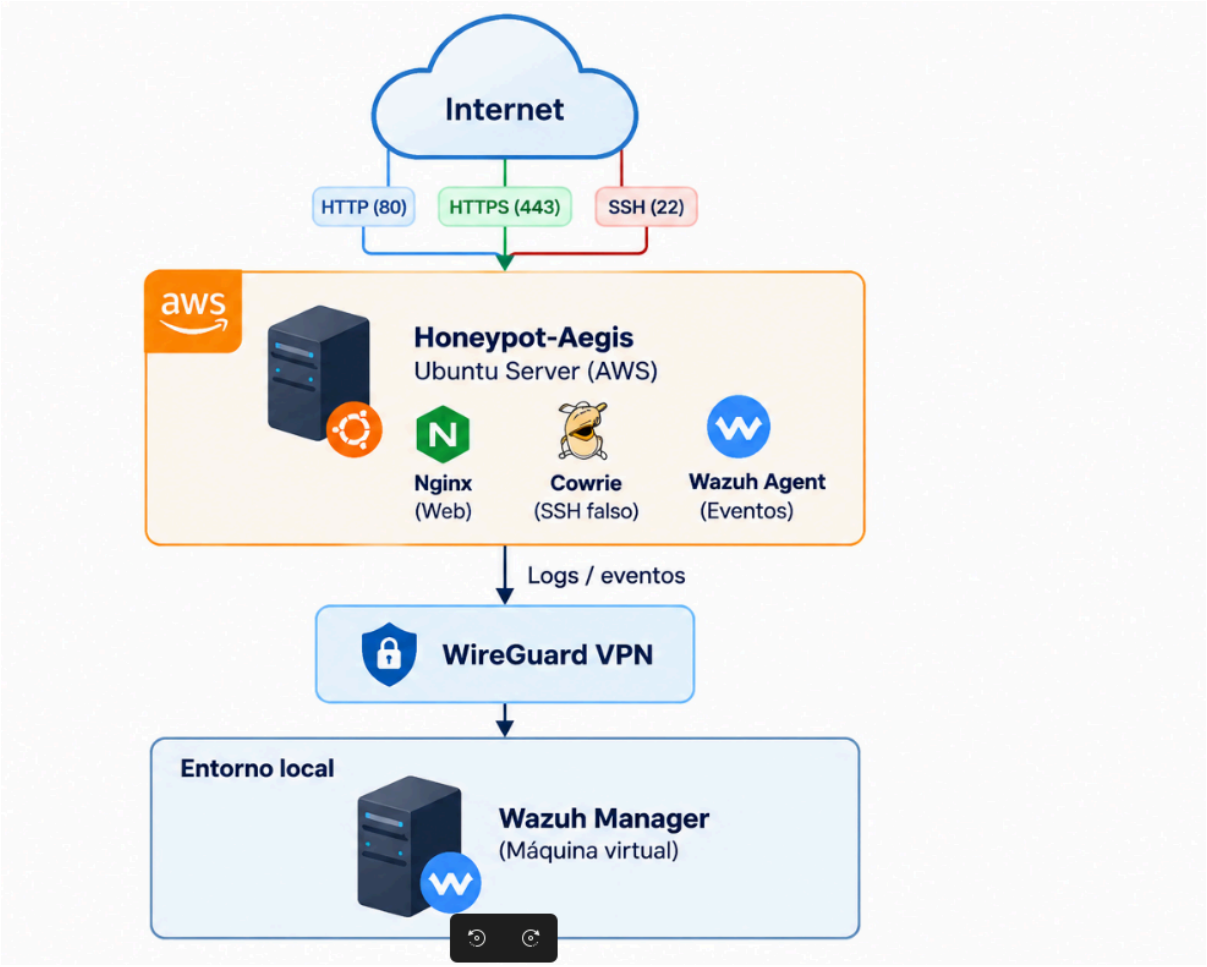
Este proyecto consiste en el despliegue de un honeypot en AWS para observar la actividad que recibe un servidor cuando algunos de sus servicios quedan expuestos a Internet.
Para ello se utilizó una instancia Ubuntu Server en AWS donde se instalaron dos servicios señuelo: Cowrie, para simular un servicio SSH, y Nginx, utilizado para publicar la página web ficticia de Aegis Bank.
Los eventos generados por el honeypot se enviaron a un servidor Wazuh instalado en una máquina virtual local. La comunicación entre AWS y el entorno local se realizó mediante WireGuard, aprovechando parte de la infraestructura creada anteriormente. De esta forma, Wazuh no quedó expuesto directamente a Internet.

1.1 Objetivo

El objetivo de esta primera parte del proyecto fue diseñar y poner en funcionamiento la infraestructura necesaria para desplegar el honeypot y centralizar sus eventos en Wazuh.
Se buscó mantener separado el servicio SSH simulado por Cowrie del acceso utilizado para administrar realmente el servidor, además de aislar el honeypot del sistema utilizado para la monitorización.
Una vez comprobado que los eventos llegaban correctamente a Wazuh, la infraestructura quedó preparada para comenzar la segunda parte del proyecto: el análisis de la actividad real recibida desde Internet.

2. Arquitectura del laboratorio

El laboratorio se desplegó utilizando AWS y un entorno local. En AWS se alojó el honeypot, formado por un servidor Ubuntu con Cowrie y Nginx, expuesto a Internet para recibir conexiones y registrar la actividad.
Para la monitorización se instaló un agente Wazuh en el honeypot. Los eventos se enviaban al Wazuh Manager, alojado en una máquina virtual del entorno local.
La comunicación entre AWS y la red local se realizó mediante una VPN WireGuard, evitando exponer directamente el servidor Wazuh a Internet.



3. Despliegue de la infraestructura en AWS

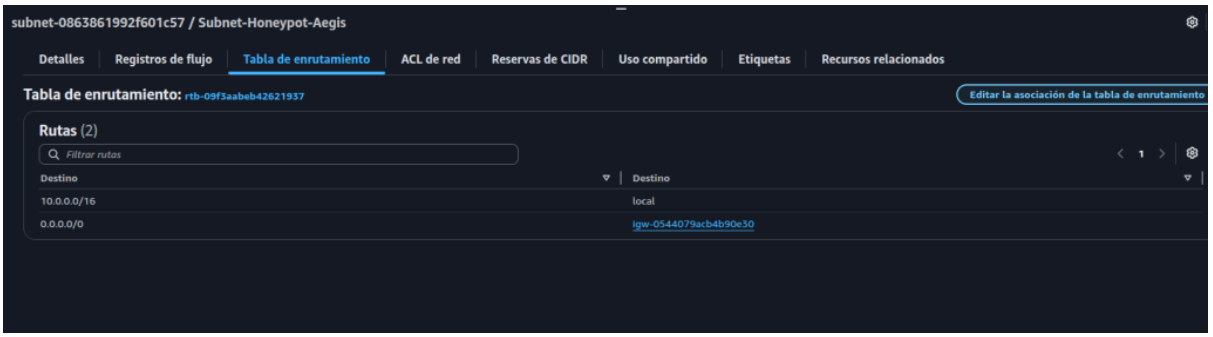
3.1 Diseño de la red

Para el laboratorio se utilizó la VPC VPC-Aegis (10.0.0.0/16), dividiendo la infraestructura en diferentes subredes según la función de cada máquina.
El honeypot se alojó en Subnet-Honeypot-Aegis (10.0.2.0/24), mientras que el servidor encargado de comunicar AWS con el entorno local mediante WireGuard se mantuvo en una subred independiente.



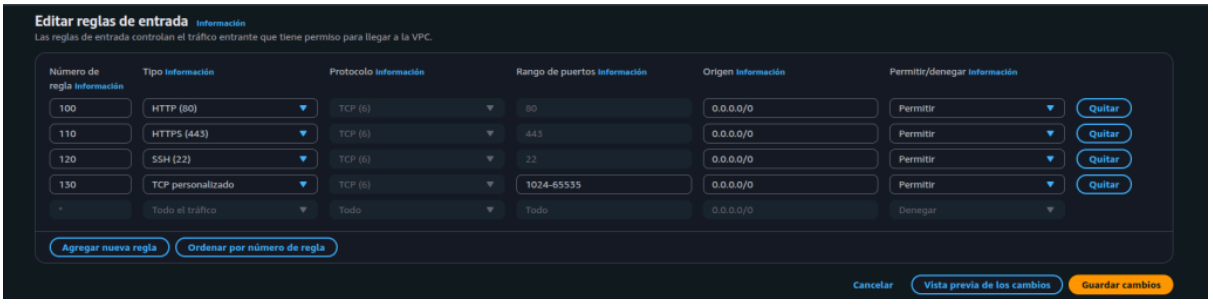
3.2 Enrutamiento y acceso a Internet

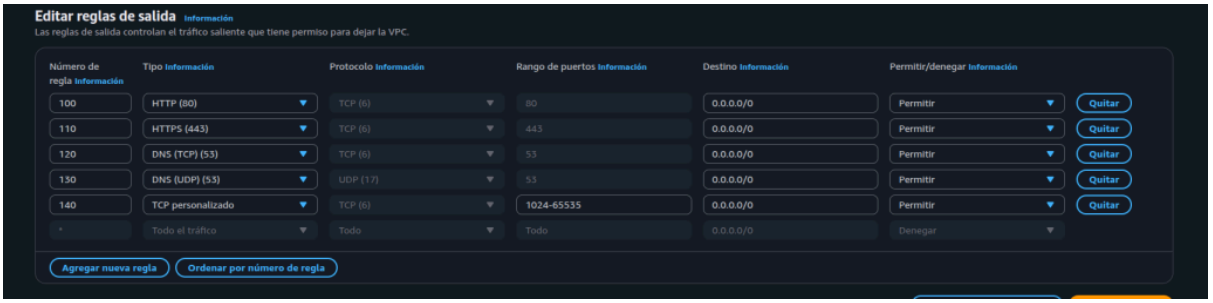
Se configuró el enrutamiento necesario para que el honeypot pudiera recibir conexiones desde Internet. La VPC disponía de salida mediante Internet Gateway y las tablas de rutas determinaban hacia dónde debía enviarse el tráfico.



3.3 Control del tráfico

Para controlar el tráfico de la infraestructura se utilizaron Network ACL (NACL) y Security Groups. La NACL se configuró a nivel de subred, mientras que los Security Groups permitieron definir el tráfico para las instancias.
En el honeypot se permitió el acceso desde Internet mediante HTTP (80), HTTPS (443) y SSH (22). El puerto 22 se utilizó para el servicio SSH señuelo de Cowrie, manteniendo separado el acceso utilizado para la administración real del servidor.





3.4 Acceso mediante VPN

Para comunicar la infraestructura de AWS con el entorno local se utilizó una VPN WireGuard. El servidor Ubuntu-Server-Aegis actuaba como punto de conexión entre ambas redes, permitiendo que el honeypot pudiera comunicarse con el servidor Wazuh instalado localmente.
Se utilizó la red VPN 10.10.10.0/24, quedando el servidor de AWS con la dirección 10.10.10.1 y el equipo local con 10.10.10.2.
De esta forma, los eventos recogidos por el agente de Wazuh instalado en el honeypot podían enviarse al Wazuh Manager del entorno local sin necesidad de exponerlo directamente a Internet.

```
mtk@mtk-IdeaPad-3-15ITL6:~$ sudo nano /etc/wireguard/wg0.conf
mtk@mtk-IdeaPad-3-15ITL6:~$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.10.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] resolvconf -a wg0 -m 0 -x
[#] wg set wg0 fwmark 51820
[#] ip -4 rule add not fwmark 51820 table 51820
[#] ip -4 rule add table main suppress_prefixlength 0
[#] ip -4 route add 0.0.0.0/0 dev wg0 table 51820
[#] sysctl -q net.ipv4.conf.all.src_valid_mark=1
[#] nft -f /dev/fd/63
mtk@mtk-IdeaPad-3-15ITL6:~$ sudo wg
interface: wg0
  public key: kNI1z1k276aZgPdkrGTpBu+nyaiUFMMcKlrHGIM17TE=
  private key: (hidden)
  listening port: 45250
  fwmark: 0xca6c

peer: 73lcEe4aKSnpg93BzfJ/slMampFQy84hF9yT3Shguj4=
  endpoint: 34.243.103.100:51820
  allowed ips: 0.0.0.0/0
  latest handshake: 10 seconds ago
  transfer: 7.93 KiB received, 6.34 KiB sent
  persistent keepalive: every 25 seconds
mtk@mtk-IdeaPad-3-15ITL6:~$
```

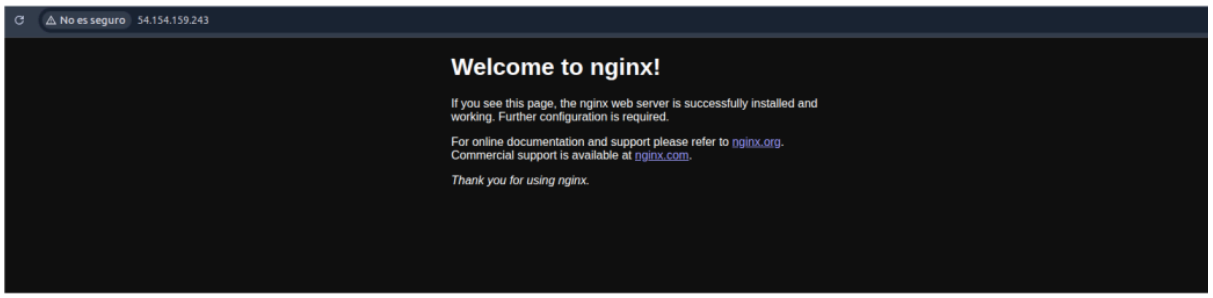
4. Despliegue y configuración del honeypot

4.1 Instalación y configuración de Nginx

El honeypot web se desplegó con Nginx en la misma instancia EC2 utilizada por Cowrie, manteniendo ambos servicios separados mediante distintos puertos.

- Puerto 22: Cowrie — servicio SSH señuelo.
- Puerto 80/443: Nginx — servicio web HTTP/HTTPS.

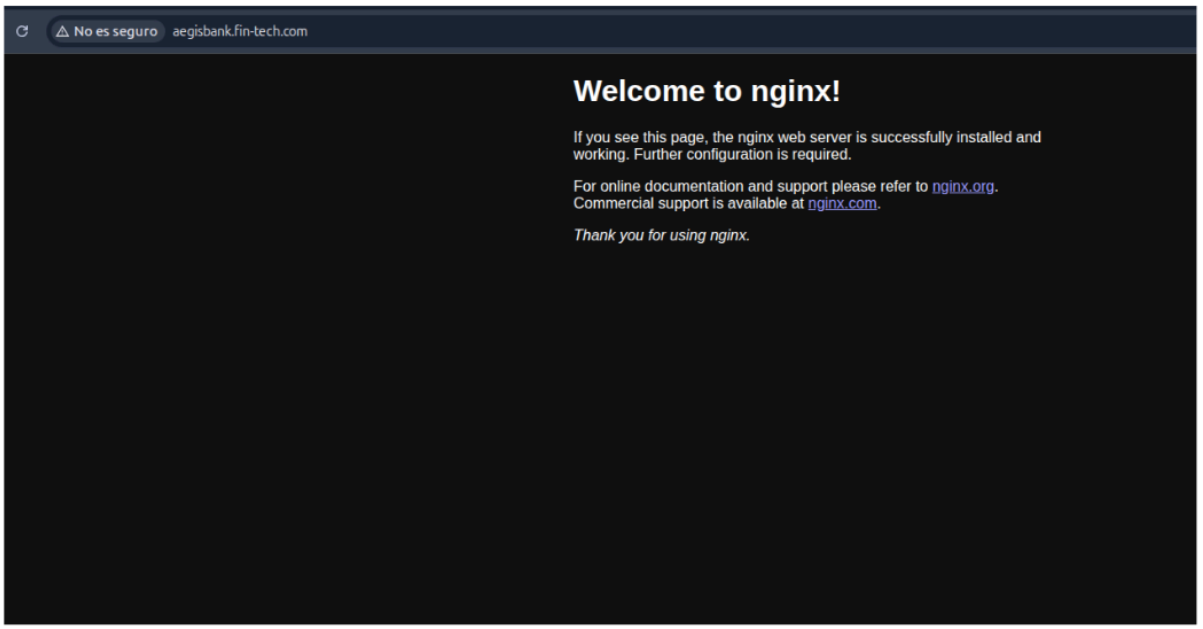
Una vez instalado Nginx, se comprobó que el servicio estaba activo y accesible desde Internet mediante las reglas de red configuradas previamente.



4.2 Configuración DNS

Para acceder a la web señuelo mediante un nombre de dominio se configuró el DNS para que el dominio utilizado en el laboratorio apuntara a la dirección pública del honeypot.
De esta forma, la web publicada mediante Nginx podía ser accesible utilizando el dominio tanto mediante HTTP como HTTPS, en lugar de utilizar directamente la dirección IP del servidor.
Una vez configurado el registro DNS como aegisbank.fin-tech.com, se comprobó la resolución mediante nslookup y el navegador.





4.3 HTTPS y certificado TLS

Para proteger las comunicaciones se habilitó HTTPS en Nginx y se obtuvo un certificado TLS de Let's Encrypt mediante Certbot para aegisbank.fin-tech.com.
Durante las primeras pruebas se detectó un error 403 Forbidden. Tras revisar la configuración y crear correctamente la página principal del sitio, se comprobó el acceso mediante HTTPS.

```
ubuntu@honeypot-aegis:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@honeypot-aegis:~$ sudo ss -lntp | grep -E '(:80|:443)'
LISTEN 0      511          0.0.0.0:443        0.0.0.0:*    users:(("nginx",pid=2253,fd=12),("nginx",pid=2252,fd=12),("nginx",pid=1747,fd=12))
LISTEN 0      511          0.0.0.0:80         0.0.0.0:*    users:(("nginx",pid=2253,fd=5),("nginx",pid=2252,fd=5),("nginx",pid=1747,fd=5))
LISTEN 0      511             [::]:443           [::]:*    users:(("nginx",pid=2253,fd=11),("nginx",pid=2252,fd=11),("nginx",pid=1747,fd=11))
LISTEN 0      511             [::]:80            [::]:*    users:(("nginx",pid=2253,fd=6),("nginx",pid=2252,fd=6),("nginx",pid=1747,fd=6))
ubuntu@honeypot-aegis:~$
```

4.4 Web señuelo Aegis Bank

Se creó una web en HTML y CSS simulando el portal de una entidad bancaria ficticia. La página se utilizó como señuelo para generar y registrar actividad web procedente de Internet.
Las peticiones recibidas por Nginx quedaron registradas para su posterior envío y análisis mediante Wazuh.



5. Despliegue del honeypot SSH con Cowrie

5.1 Instalación y configuración de Cowrie

Para crear el servicio SSH señuelo se instaló Cowrie utilizando un usuario específico y un entorno virtual de Python, manteniendo sus dependencias separadas de las del sistema.
La configuración se realizó mediante el archivo cowrie.cfg, donde se definió bank-server como nombre del sistema mostrado a las conexiones recibidas.
Una vez iniciado el servicio, se comprobó que Cowrie estaba escuchando correctamente en el puerto 2222.

```
(cowrie-env) cowrie@honeypot-aegis:~/cowrie$ cowrie --help
usage: cowrie [-h] {init,start,stop,force-stop,restart,status,shell,bash,sh} [args ...]

cowrie honeypot service manager

positional arguments:
  {init,start,stop,force-stop,restart,status,shell,bash,sh}
                        Command to execute
  args                  Additional arguments to pass to twistd

options:
  -h, --help            show this help message and exit
```

```
(cowrie-env) cowrie@honeypot-aegis:~/cowrie$ ss -lntp | grep -E '(:22|:2222)'
LISTEN 0      4096         0.0.0.0:22        0.0.0.0:*
LISTEN 0      50           0.0.0.0:2222      0.0.0.0:*    users:(("twistd",pid=2704,fd=11))
LISTEN 0      4096            [::]:22           [::]:*
```

5.2 Separación del SSH real y el servicio señuelo

Para evitar que el servicio SSH real utilizado para administrar Ubuntu quedara expuesto en el mismo puerto que el honeypot, se trasladó OpenSSH del puerto 22 al 22222.
Durante la configuración se detectó que Ubuntu Server 24.04 también gestionaba SSH mediante ssh.socket, que continuaba escuchando en el puerto 22. Fue necesario modificar también esta configuración antes de conseguir que el SSH administrativo quedara únicamente en el puerto 22222.
Antes de cerrar la conexión existente, se comprobó el acceso administrativo mediante el nuevo puerto a través de la VPN WireGuard.

```
ubuntu@honeypot-aegis:~$ sudo systemctl restart ssh.socket
ubuntu@honeypot-aegis:~$ sudo ss -lntp | grep -E '(:22|:2222|:22222)'
LISTEN 0      4096         0.0.0.0:22222     0.0.0.0:*    users:(("sshd",pid=2884,fd=3),("systemd",pid=1,fd=70))
LISTEN 0      50           0.0.0.0:2222      0.0.0.0:*    users:(("twistd",pid=2704,fd=11))
LISTEN 0      4096            [::]:22222        [::]:*    users:(("sshd",pid=2884,fd=4),("systemd",pid=1,fd=85))
```

5.3 Redirección del puerto 22 hacia Cowrie

Cowrie permaneció escuchando internamente en el puerto 2222, mientras que las conexiones SSH procedentes de Internet debían acceder mediante el puerto estándar 22. Para ello se configuró una regla de iptables que redirigía las conexiones recibidas en el puerto 22 hacia Cowrie, así, una conexión externa al puerto 22 era atendida por Cowrie, mientras que el SSH real permanecía separado en el puerto 22222.

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT
--to-ports 2222
```

5.4 Persistencia y arranque automático

Para mantener la configuración después de reiniciar la instancia se utilizó iptables-persistent, conservando la regla de redirección del puerto 22.
Cowrie también se configuró como un servicio de systemd para que se iniciara automáticamente con el servidor.

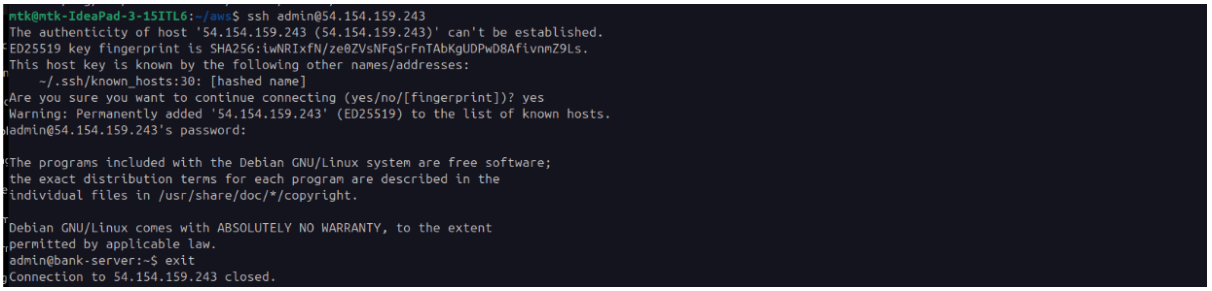
Durante las primeras pruebas, el servicio no encontraba twistd porque se encontraba instalado dentro del entorno virtual de Python. El problema se solucionó añadiendo al servicio la ruta correspondiente al entorno virtual.

```
Environment="PATH=/home/cowrie/cowrie/cowrie-env/bin:/usr/local/sb
in:/usr/local/bin:/usr/s
bin:/usr/bin:/sbin:/bin"
```



5.5 Validación desde Internet

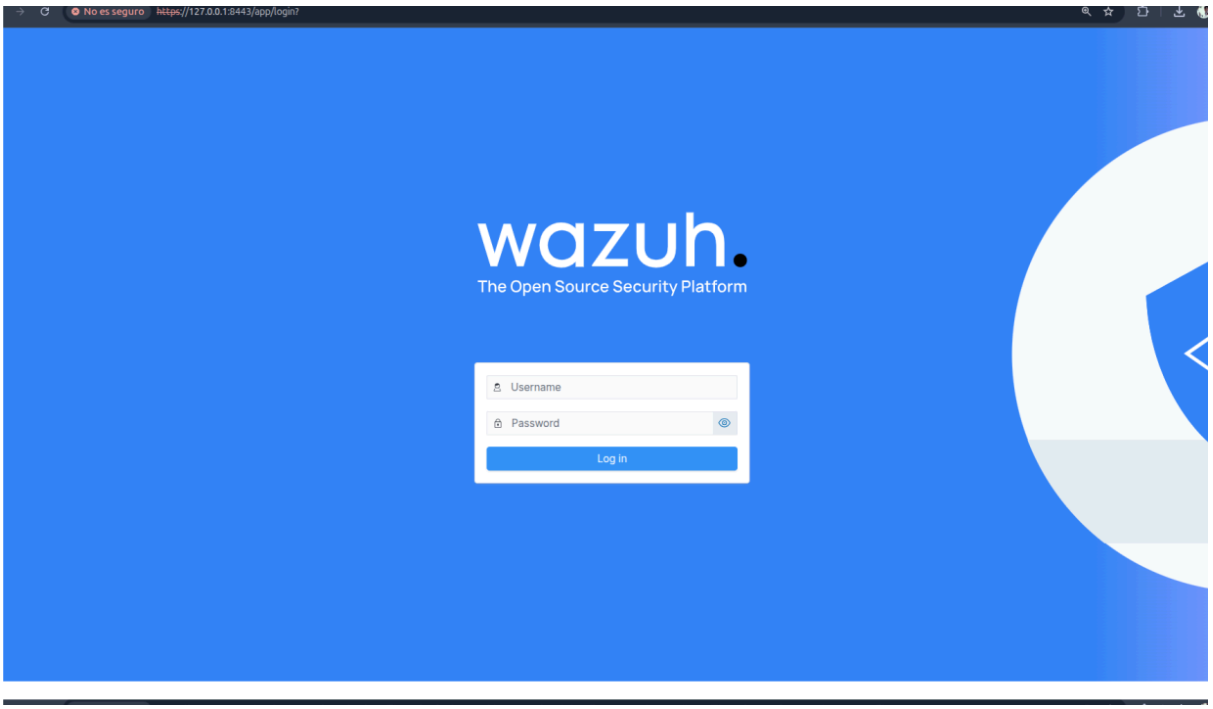
Finalmente, se realizó una conexión SSH desde una red externa hacia la dirección pública del honeypot utilizando el puerto 22.
La conexión fue atendida por Cowrie y posteriormente se comprobó en sus registros que tanto el intento de autenticación como la actividad realizada dentro del sistema señuelo habían quedado registrados.



6. Integración con Wazuh

6.1 Servidor Wazuh local

Para centralizar la monitorización se creó una máquina virtual independiente con Ubuntu Server 24.04 LTS, manteniendo Wazuh en el entorno local y separado del honeypot desplegado en AWS.
Se realizó una instalación all-in-one de Wazuh, incluyendo Wazuh Manager, Indexer y Dashboard en el mismo servidor. Una vez terminada la instalación, se comprobó el acceso al Dashboard desde el equipo anfitrión.

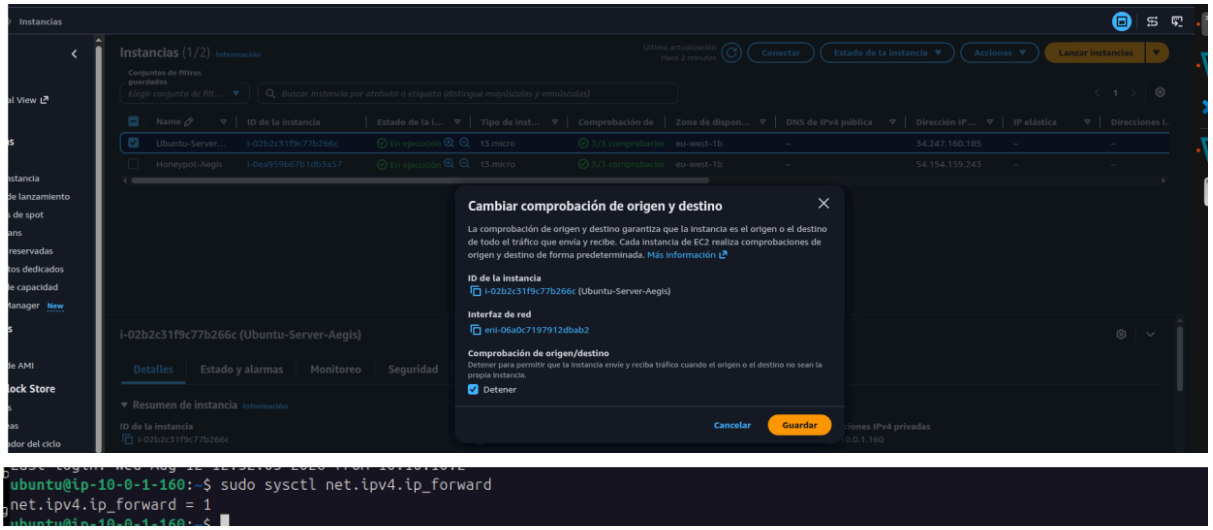


6.2 Comunicación entre AWS y Wazuh

Para comunicar el honeypot de AWS con el servidor Wazuh local se reutilizó la VPN WireGuard configurada anteriormente. Ubuntu-Server-Aegis actuaba como intermediario entre la red de AWS y el equipo local, evitando exponer directamente Wazuh a Internet



Para permitir este recorrido se añadió en AWS una ruta hacia la red WireGuard 10.10.10.0/24 utilizando Ubuntu-Server-Aegis como intermediario. También se habilitó el reenvío de paquetes y se desactivó Source/Destination Check en la instancia que actuaba como router.



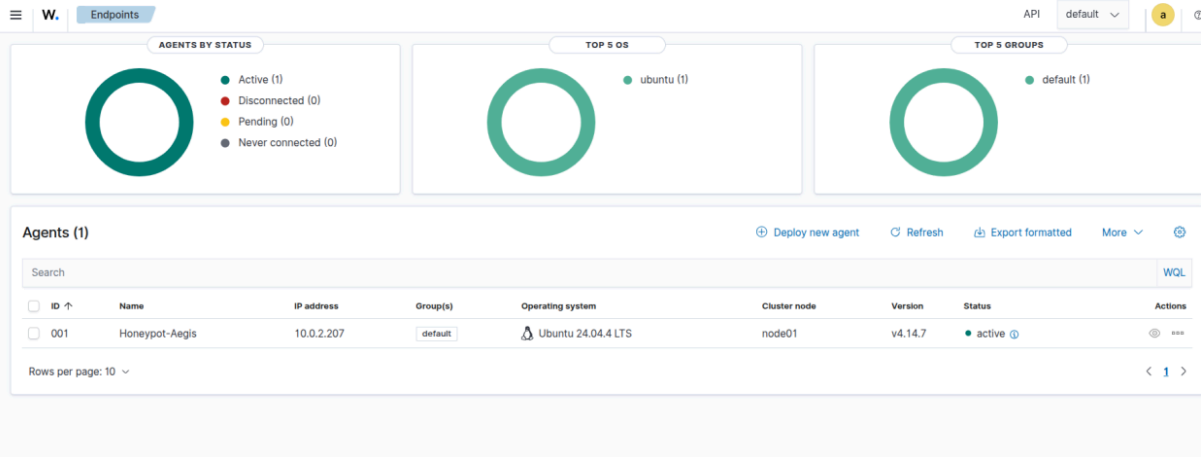
Durante las pruebas se detectó además un conflicto entre la subred del honeypot 10.0.2.0/24 y la red utilizada inicialmente por VirtualBox. Para solucionarlo, la red de Wazuh se cambió a 192.168.50.0/24, quedando Wazuh Manager con la dirección 192.168.50.15



6.3 Conexión del agente Wazuh

Una vez preparada la comunicación entre AWS y el entorno local, se instaló el agente de Wazuh en Honeypot-Aegis y se configuró para comunicarse con Wazuh Manager a través de WireGuard.

Para realizar el registro se utilizó la dirección 10.10.10.2, correspondiente al extremo local de la VPN. Una vez finalizado, se comprobó que Honeypot-Aegis aparecía correctamente registrado en Wazuh con el identificador 001



7. Resultado final

Una vez terminada la configuración, el laboratorio quedó formado por un honeypot desplegado en AWS con dos servicios señuelo: Cowrie para SSH y Nginx para el servicio web.

El acceso administrativo al servidor quedó separado del servicio SSH expuesto y la comunicación entre AWS y el entorno local se realizó mediante WireGuard, manteniendo la infraestructura de monitorización fuera de Internet.

Las pruebas realizadas permitieron comprobar que tanto el servicio web como Cowrie eran accesibles desde Internet y que el honeypot estaba preparado para registrar la actividad recibida.

8. Conclusiones

Este proyecto permitió poner en práctica diferentes conocimientos de administración de sistemas y redes, combinando AWS, Linux, Nginx, DNS, HTTPS, Cowrie, firewalls y WireGuard dentro de una misma infraestructura.

Durante el despliegue también aparecieron algunos problemas, como la configuración de ssh.socket o el arranque automático de Cowrie mediante systemd, que fue necesario revisar y solucionar para conseguir que los servicios funcionaran bien.

Una vez terminada la infraestructura, el honeypot quedó preparado para comenzar la siguiente fase del proyecto: recoger y analizar mediante Wazuh la actividad recibida desde Internet.